

PROJECT BRIEFING · 00 & 00 00



GPS 00 00000 00 0 0

— 00 HTML 000 0000 00 · 00 · 00 · 00 0000 00 000 00 00



000, 000, 000

OVERVIEW

이제부터 시작

“이제부터 시작, 언제 시작?” — GPS 이 시작 버튼을 누르면 시작 버튼을 누르면



이제부터 시작

가, 이 시작 버튼을 누르면 시작 버튼을 누르면



이제부터 시작

이제부터 시작 버튼을 누르면 /24 시작 / 시작 버튼을 누르면



이제부터 시작

이제부터 시작 버튼을 누르면 시작 버튼을 누르면 시작 버튼을 누르면



이제부터 시작

가, 이 시작 버튼을 누르면 시작 버튼을 누르면



이제부터 시작

이제부터 시작 버튼을 누르면 시작 버튼을 누르면 시작 버튼을 누르면



이제부터 시작

이제부터 시작 버튼을 누르면 시작 버튼을 누르면 / 시작 버튼을 누르면

TECH STACK



이 프로젝트는 HTML, CSS, JavaScript, Leaflet.js, Geolocation API, localStorage 등을 사용합니다.



기술 스택

HTML · CSS · Vanilla JavaScript

이 프로젝트는 HTML (index.html) 파일에서 CSS와 JavaScript를 포함합니다.

기술 스택

Leaflet.js 1.9.4 + OpenStreetMap

이 프로젝트는 OpenStreetMap (OSM) 데이터를 CartoDB에서 가져옵니다.



기술 스택

Geolocation API

navigator.geolocation API를 사용하여 GPS 데이터를 가져옵니다.



기술 스택

localStorage (브라우저에 저장)

이 프로젝트는 localStorage를 사용하여 데이터를 JSON 형식으로 저장합니다.

DATA STRUCTURE

TOILETS

TOILETS 데이터 구조는 다음과 같습니다. 각 TOILETS 데이터는 다음과 같습니다.

TOILETS 데이터: TOILETS 데이터 — 'TOILETS 데이터' 데이터 (TOILETS 데이터 데이터).

```
const TOILETS = [  
  { id: 1, name: "TOILETS 2 TOILETS",  
    lat: 37.4984, lng: 127.0260,  
    type: "public", rating: 4.2, reviews: 38,  
    bidet: true, open24h: true, disabled: true,  
    clean: 85, stalls: 6, loc: "TOILETS 1" },  
  // ... TOILETS 4000 TOILETS TOILETS  
];
```

lat / lng

TOILETS 데이터 데이터

type

"public"(TOILETS) / "secret"(TOILETS)

rating / reviews

TOILETS 데이터 데이터

bidet / open24h / disabled

TOILETS · 24 TOILETS · TOILETS TOILETS → TOILETS TOILETS

clean / stalls / loc

TOILETS (%) · TOILETS · TOILETS TOILETS

가 TOILETS 데이터 dynToilets 데이터 데이터, TOILETS 데이터 TOILETS 데이터 데이터.

MAP RENDERING

□□ & □□ □□ □□

Leaflet □ divIcon □ HTML □□□ □□□ □□□□ □□□□□ .

```
function toiletIcon(type, beta) {  
  const c = type === 'public' ? '#2979ff' : '#ffc107';  
  const e = type === 'public' ? '□' : '□';  
  return L.divIcon({ html: `  
    <div style="width:38px;height:38px;  
      background:${c};border-radius:50% 50% 50% 0;  
      transform:rotate(-45deg)">  
      <span> ${e}</span></div> ` });  
}
```

□□ □□

- ① TOILETS + dynToilets □□□ □□ □□ □□□ □□□□ □□□
- ② □ □□□□ type □ □□ toiletIcon() □ □□ / □□ □□□ □□□ □□
- ③ L.marker(...).addTo(map) 가 □□ □□□□□ □□□□ □□ □□ (openSheet) □□
- ④ 가 □□□ □□□ draggable: true 가 □□□□ □□ □□ □ □□□□□

GEOLOCATION & DISTANCE

위치 정보 & 거리 정보

Geolocation API는 GPS 정보와, Haversine) 알고리즘을 사용하여 두 위치 간의 거리를 계산합니다.

```
function dist(a, b, c, d) {  
  const R = 6371000; // 지구 반지름 (m)  
  const f1 = a * Math.PI / 180, f2 = c * Math.PI / 180;  
  const df = (c - a) * Math.PI / 180, dl = (d - b) * Math.PI / 180;  
  const x = Math.sin(df / 2) ** 2  
    + Math.cos(f1) * Math.cos(f2) * Math.sin(dl / 2) ** 2;  
  return R * 2 * Math.atan2(Math.sqrt(x), Math.sqrt(1 - x));  
}  
function fmtD(m) { return m < 1000 ? `${Math.round(m)}m`  
  : `${(m / 1000).toFixed(1)}km`; }
```

코드 예제

- ① navigator.geolocation.watchPosition() 함수를 사용하여 위치를 추적합니다 (uLat, uLng) 값을 반환합니다.
- ② 이 코드는 '위치 정보'라는 제목의 페이지를 생성합니다 (HTTPS 주소, 이 GitHub Pages에 호스팅됩니다).
- ③ dist() 함수는 두 위치 간의 거리 (m)를 반환합니다.
- ④ fmtD() 함수는 거리를 "240m" / "1.3km" 형식으로 포맷팅합니다.

FILTER & SEARCH

□□ · □□ □□

□□ □□ □□□□ □□ □□□ □□□ □□ □□□□ '□□ □□□' □□□□ □□□□□□ .

```
function passFilter(t) {  
  switch (activeFilter) {  
    case 'open24h': return t.open24h;  
    case 'disabled': return t.disabled;  
    case 'public': return t.type === 'public';  
    case 'secret': return t.type === 'secret';  
    default: return true; // '□□ '   
  }  
}
```

□□ □□

- ① □□ □ (□□ /24 가ㄴ / □□□□ / □□ / □□) □ □□□ activeFilter □□ □□
- ② renderMarkers() 가 □□ □□□ □□□ passFilter() □ □□ □□□ □□ □□
- ③ □□□ □□ □□ □ □□ □□ □□ □ □□□ □□ · □□ □□□ □□□ □□□□ □ □□ □□
- ④ □□ □□ □ map.setView() □ □□□ □□ □□□ □□□□ □□ □□□ □

SECRET TOILET & COIN

🚻 🚽 & 🪙 🚻

🚻 1 🪙 🚻🚻🚻 🚻 🚻🚻🚻 . 🚻 🚻 🚻 . `unlockedIds` 🚻🚻🚻 🚻 🚻🚻 🚻🚻🚻 .

```
function useCoin(){
  if(coins<=0){ show Toast('🚻🚻 🚻🚻🚻 '); return; }
  coins--; localStorage.setItem ('coins', coins);

  // 🚻 🚻🚻 🚻 → 🚻🚻 🚻 🚻🚻🚻 🚻
  if(!unlockedIds.includes(sel.id)){
    unlockedIds.push(sel.id);
    localStorage.setItem ('unlockedIds',
      JSON.stringify(unlockedIds));
  }
  // 🚻🚻🚻 . 🚻 🚻 🚻🚻 🚻
}
```

🚻 🚻

- ① 🚻 🚻🚻 🚻🚻🚻 “🚻 1 🪙 🚻🚻🚻 🚻” 🚻🚻 🚻🚻 🚻🚻 🚻
 - ② `useCoin()` 🚻 🚻🚻 1 🪙 🚻🚻🚻 , 🚻🚻🚻 . 🚻 🚻🚻 🚻
 - ③ 🚻🚻 🚻🚻 `id` 🚻 `unlockedIds` 🚻 🚻 → 🚻🚻 🚻 🚻 🚻🚻 🚻 🚻🚻 🚻
 - ④ 🚻🚻🚻 “🚻 / 🚻” 🚻 🚻🚻 🚻 🚻🚻🚻 `display` 🚻 🚻 — 🚻🚻 🚻🚻 🚻🚻 🚻
- 🚻 🚻 : 🚻 🚻🚻🚻 🚻🚻🚻 🚻 → 🚻 +1, 🚻🚻🚻 🚻 🚻 🚻🚻 🚻
(submittedPwIds)

REVIEWS & PERSISTENCE

리뷰 저장 & 로컬 저장 (localStorage)

리뷰 객체 reviewsDB 로컬에 저장, 평균 별점 계산 로컬에 저장.

```
function computeRating(t){
  const uRevs = reviewsDB[String(t.id)] || [];
  if(!uRevs.length) return { avg:t.rating, cnt:0 }; // 첫 리뷰는 평균 별점 0

  const uAvg = uRevs.reduce((a,b) => a + b.stars, 0) / uRevs.length;
  return { avg: Math.round(uAvg * 10) / 10, cnt: uRevs.length };
}
```

주요 로직

- ① id · stars · cnt (총 4MB) 로컬에 저장 reviewsDB[id]에 저장
- ② localStorage.setItem('reviews', JSON.stringify(reviewsDB)) 로 로컬에 저장
- ③ computeRating() 가 첫 리뷰는 평균 별점 0 — 리뷰가 추가되면 평균 별점 계산 (cnt:0)
- ④ 리뷰 수 cnt +1 로컬에 저장
→ 로컬에 저장된 리뷰 객체 업데이트

WRAP-UP



이제 HTML 이 아닌 SPA

Leaflet 이나 · · · · · 을 사용하여 지도를 표시할 수 있습니다.



이제 localStorage

이제 localStorage를 사용하여 데이터를 저장할 수 있습니다.



이제 localStorage

이제 localStorage를 사용하여 데이터를 저장할 수 있습니다.



이제 localStorage

- 이 API를 사용하여 데이터를 저장할 수 있습니다.
- DB를 사용하여 데이터를 저장할 수 있습니다.
- 이 API를 사용하여 데이터를 저장할 수 있습니다.

가 이 API를 사용하여 데이터를 저장할 수 있습니다. (GitHub Pages)